

Enterprise Phase 2.5B

CRM Opportunity Frontend Integration & End-to-End Validation

Binding Implementation Specification for Claude Code

STATUS: BINDING

Phase 2.5A Authentication Frontend is complete.

Phase 2 CRM Opportunity Commands & API are complete.

This phase connects the React frontend to the real backend and validates the full Opportunity workflow without duplicating backend business rules in the browser.

- Primary goal: a user can list, create, inspect, edit, progress, reassign, win, and lose Opportunities from the React application through authoritative backend contracts.
- Secondary goal: prove that Phase 1.5 authorization and Phase 2 command/query contracts work correctly in a real browser flow.
- Non-goal: build Customer Need Intelligence, Quotes, Activities, Feedback, Consent, AI agents, or analytics in this phase.

FOUNDATION-FIRST. BACKEND-AUTHORITATIVE. NO FRONTEND DOMAIN FORKS.

1. Mission and Phase Boundary

This session is dedicated ONLY to PHASE 2.5B - CRM Opportunity Frontend Integration & End-to-End Validation.

Previous phases are binding foundations and must be consumed, not redesigned:

- Phase 0.5 - MasterData / Party Foundation - COMPLETE
- Phase 1 - CRM Lifecycle / Pipeline Foundation - COMPLETE
- Phase 1.5 - Enterprise Access / Authorization Foundation - COMPLETE
- Phase 2 - Opportunity Commands & API - COMPLETE
- Phase 2.5A - Authentication Frontend - COMPLETE

The frontend must integrate with the real Phase 2 API and the real Phase 1.5 authorization decisions. Do not create browser-only business rules, mock-only flows, parallel permission models, or temporary API contracts.

2. First Rule - Inspect Before Editing

Do not start implementation from this document alone. First inspect the actual repository and verify the current contracts.

- React application structure, routing, shell, layouts, state management, query client, form stack, i18n, notification/toast system, and design system.
- Phase 2.5A authentication implementation: session restore, current principal, tenant context, protected routes, 401/403 handling, tenant switching.
- Phase 2 Opportunity command endpoints, query endpoints, DTOs, ProblemDetails/error contracts, concurrency contract, idempotency contract, available-actions or transition projections if implemented.
- Phase 1.5 authorization contracts: coarse capability, record-level policy, field-level READ/WRITE, obligations/approval requirements, masking rules, denial behavior.
- Pipeline and stage endpoints/configuration from Phase 1.
- Existing frontend mocks/MSW handlers and whether they should be removed, disabled, or retained only for isolated component tests.
- Existing tests and implementation reports from Phase 2 and Phase 2.5A.

The repository is the source of truth for implemented behavior. Binding architecture decisions are the source of truth for intended behavior. If they conflict, record the conflict before changing code.

3. Required Planning Artifact

Before production code changes, create:

```
docs/architecture-analysis/PHASE_2_5B_CRM_FRONTEND_IMPLEMENTATION_PLAN.md
```

The plan must include:

1. Verified frontend and backend repository state.
2. Actual Phase 2 API surface and contract inventory.
3. Authentication and tenant-context integration points from Phase 2.5A.
4. Opportunity list/detail/create/edit/stage/reassign/win/lose UX flow.
5. Authorization/FLS/obligation consumption strategy.
6. Available-actions and allowed-transitions strategy.
7. Concurrency and stale-write UX.
8. Idempotent-submit and duplicate-click strategy.
9. Error taxonomy and user-facing behavior.
10. Loading, empty, unauthorized, forbidden, stale, and retry states.
11. Exact files expected to change/add.
12. Playwright and component/integration test plan.
13. Explicit non-goals and any OPEN DECISION items.

If there is no blocking OPEN DECISION, continue implementation without waiting for approval. If a binding architectural decision is missing, stop and ask only for that decision.

4. Definition of Done

Phase 2.5B is complete only when a real authenticated user can execute the Opportunity workflow against the real backend:

```
Opportunity List
-> Create Opportunity
-> Open Opportunity Detail
-> Edit allowed fields
-> Change Pipeline Stage
-> Reassign (if authorized)
-> Win OR Lose
-> Observe final state and audit-safe API behavior
```

The flow must respect tenant context, record-level authorization, field-level security, policy obligations, optimistic concurrency, idempotency, and backend domain invariants.

5. Frontend Route and Information Architecture

Use the existing shell architecture. CRM is an application surface with its own internal navigation; do not redesign the global shell in this phase.

Expected route concepts, subject to existing route conventions:

```
/crm/opportunities
/crm/opportunities/new
/crm/opportunities/:id
```

Do not create redundant routes for every command unless the existing frontend architecture requires them. Stage change, reassign, win, and lose are normally actions inside the Opportunity detail experience.

5.1 Opportunity List

The list must be a usable work surface, not a decorative dashboard.

- Fetch real Opportunity data from Phase 2 query API.
- Respect tenant context from Phase 2.5A.
- Show only fields the backend permits the caller to read.
- Support loading, empty, error, unauthorized/forbidden, and refresh states.
- Prefer existing table/list primitives and saved-view conventions if already present.
- Do not infer hidden records or counts from authorization-denied responses.
- Do not build reporting, charts, forecasting, or analytics in this phase.

5.2 Opportunity Detail

The detail page is the authoritative frontend workspace for one Opportunity. It should render server state plus server-authorized actions.

- Lifecycle status.
- Current pipeline and stage.
- Party/customer reference and allowed customer display fields.
- Assigned principal/owner.
- Editable CRM-owned fields implemented in Phase 2.
- Row version / concurrency token kept internally for mutation requests.
- Available actions and allowed target stages, if exposed by the backend.
- Field-level READ masking/omission behavior exactly as defined by backend contracts.

6. Backend-Authoritative UI Rule

The React application must not reproduce domain or authorization logic that already belongs to the backend.

Forbidden pattern:

```
if (opportunity.lifecycleStatus === 'Open') { showWinButton(); }
```

Preferred pattern:

```
availableActions.canWin === true
allowedStageTransitions = [...]
fieldAccess.estimatedAmount.write === true
```

The frontend may use these projections to render UX, but every command must still be re-authorized and re-validated by the backend at execution time.

7. Required User Flows

Flow	User intent	Binding rule
List	Browse Opportunities in current tenant	Real query API; no mock-only data
Create	Create a valid Opportunity	Use Phase 2 create contract and initial lifecycle/stage semantics
Detail	Open one Opportunity	Tenant-safe query + record policy + field READ
Edit	Change editable CRM-owned fields	Field WRITE + concurrency + evidence as backend defines
Stage	Move to an allowed pipeline stage	Only allowed target stages; backend revalidates
Reassign	Change assigned principal	Only when backend says allowed; no arbitrary principal assumptions
Win	Close as Won	Canonical lifecycle action, not a UI status patch
Lose	Close as Lost	Use backend lost contract; do not map generic cancel blindly

8. Create Opportunity

Build the create form from the actual Phase 2 contract. Do not add fields merely because they might be useful later.

The form must:

- Use React Hook Form / Zod or the repository-standard form stack.
- Mirror client-side validation only for immediate UX; backend remains authoritative.
- Resolve tenant context from the authenticated session, never from an editable browser field.
- Use the backend-defined initial lifecycle and entry/default pipeline-stage semantics.
- Prevent duplicate submissions while still relying on server idempotency.
- Generate/pass idempotency keys according to the Phase 2 contract.
- Handle validation errors at field/form level without exposing internal exception details.
- After success, navigate to the canonical Opportunity detail route or the repository-established success path.

9. Edit Opportunity

Do not implement a generic unrestricted PATCH UI. Only expose fields that Phase 2 explicitly allows to be mutated.

- Respect field-level WRITE authorization.
- Do not render an editable control for a field that the caller cannot write.
- If a field is readable but not writable, render it read-only.
- If a field is not readable, omit or mask it according to the backend policy contract.
- Send expected row/version state using the exact Phase 2 concurrency convention.
- Never silently refresh and overwrite a concurrent change.

10. Pipeline Stage Change

Pipeline stage is tenant-configurable. The frontend must not hard-code named stages such as Waiting, Offered, Proposal, or Negotiation.

The stage-change UI must be driven by backend data:

```
currentPipelineStage
allowedTargetStages[]
transition metadata / obligations if exposed
```

- Only show/select valid targets returned by the backend.
- Do not assume every stage can move to every other stage.
- Do not select the first stage by sort order as a fallback.
- Do not equate a stage label such as "Teklif Verildi" with Quote ownership or lifecycle Won.
- If the backend reports an approval/obligation, surface the state rather than bypassing it.

11. Won and Lost Actions

Won/Lost are canonical lifecycle outcomes. They are commands, not arbitrary status edits.

Won action:

- Show only when backend-authorized and domain-valid.
- Use the dedicated Phase 2 command.
- Include expected version and idempotency metadata as required.
- After success, refetch or apply server response as the new canonical state.

Lost action:

- Use the actual Phase 2 lost-reason contract.
- If a structured reason is required, use backend-provided options.
- If free text is supported, validate only according to backend contract.
- Do not treat Cancel and Lost as identical unless Phase 2 explicitly does so.

12. Reassign Opportunity

Reassignment is security-sensitive and must consume Phase 1.5 record/field authorization.

- Do not expose arbitrary principals unless the backend provides an eligible-assignee query.
- Do not infer team/org/territory membership in the browser.
- If Phase 2 has no safe eligible-assignee endpoint, mark the UX dependency explicitly rather than querying unrelated user data.
- Reassignment must enforce expected version/concurrency if the backend contract requires it.
- Policy obligations or approval requirements must be surfaced, not flattened into allow/deny.

13. Authorization, Field Security, and Obligations

Phase 2.5B must consume the Phase 1.5 authorization model exactly. It must not create a parallel frontend permission model.

13.1 Record-Level Authorization

A user may have the global capability to view/edit Opportunities but still be denied a specific record due to ownership, team, org unit, territory, lifecycle, amount, or other policy attributes. Treat backend record policy as authoritative.

13.2 Field-Level READ

Do not send or display sensitive values merely because the component can hide them visually. If the backend omits/masks a field, the UI must preserve that meaning and must not attempt to reconstruct the value from another endpoint.

13.3 Field-Level WRITE

Editability is not equivalent to record-level edit permission. Controls must reflect field-write access where the backend exposes it.

13.4 Obligations / Approval

If authorization returns RequireApproval or another obligation, the frontend must represent that state explicitly. Do not silently execute a direct command or invent a local approval flow. If Phase 2 exposes an obligation contract but no approval UI is yet implemented, show a clear blocked/pending state consistent with the existing design system.

14. Available Actions and Transitions

If Phase 2 exposes OpportunityCapabilities / AvailableActions / AvailableTransitions, this projection should drive action visibility and enablement.

```
{
  "canEdit": true,
  "canReassign": false,
  "canWin": true,
  "canLose": true,
  "allowedTargetStages": [ ... ]
}
```

If the backend does not expose an authoritative action projection, do not duplicate complex logic in React. Record this as an integration gap and use the smallest safe alternative supported by current contracts.

15. Concurrency - Mandatory UX

Optimistic concurrency is a user-visible behavior, not only a backend detail.

Scenario:

```
User A loads version 5
User B updates -> version 6
User A submits version 5
Backend -> concurrency conflict
```

Required frontend behavior:

- Do not overwrite version 6.
- Show a clear conflict message.
- Offer a safe refresh/reload of the latest server state.
- Preserve unsaved user input where practical, but do not auto-merge fields unless a binding merge strategy exists.
- Do not automatically retry a stale mutation with a new version.

16. Idempotency and Duplicate Submission UX

Frontend controls may disable while submitting, but backend idempotency remains mandatory for retrievable mutations.

- Create one idempotency key per logical user action according to Phase 2 convention.
- A network retry of the same logical action must reuse the same key.

- Do not generate a new key merely because Axios retried the request.
- An exact replay must not create duplicate Opportunity state, Evidence, or Outbox messages.
- If the backend returns an idempotency-key-reuse conflict, surface it as a conflict, not as a generic server error.

17. Error and Empty-State Contract

Condition	Required UX
400 / validation	Show field or form validation; keep user input.
401	Use Phase 2.5A session recovery/refresh flow; if recovery fails, return to login.
403	Show forbidden state; do not pretend the record does not exist unless backend intentionally collapses it.
404	Show not-found/inaccessible behavior exactly as backend contract defines; avoid existence leaks.
409 concurrency	Show stale-data conflict and refresh option.
409 idempotency	Show logical request conflict; do not auto-resubmit with new key.
422/domain rule if used	Show business-rule explanation from safe ProblemDetails fields.
5xx/network	Show retry-safe generic failure; do not expose stack traces or internals.

18. React Data and State Rules

Use existing repository conventions. Do not create a second state-management layer.

- Server state: TanStack Query or the current established query client.
- Auth/session/tenant: reuse Phase 2.5A store/context; do not persist access JWT in localStorage/sessionStorage.
- Forms: reuse current form stack.
- After mutation: update from authoritative server response or invalidate/refetch the minimal affected queries.
- Avoid global Zustand state for normal server entities unless an established repository pattern explicitly requires it.
- Query keys must include tenant-sensitive identity where necessary so tenant switching cannot display stale cross-tenant data.
- Cancel/invalidate tenant-bound requests/cache on tenant switch according to Phase 2.5A conventions.

19. Loading and Interaction Quality

Enterprise UX requires explicit intermediate states.

- Initial page loading.
- Background refetch.
- Mutation pending.
- Disabled action with explanation when appropriate.
- Empty Opportunity list.
- No allowed stage transitions.
- Forbidden record/action.
- Concurrency conflict.
- Session expired and restored.

- Network failure and retry.
- Terminal Won/Lost state.

Do not use optimistic updates for lifecycle/authorization-sensitive mutations unless Phase 2.5B can guarantee a safe rollback and the repository already has a binding optimistic-update pattern. Default to server-confirmed updates for Stage, Reassign, Win, and Lose.

20. Mocking and Real API Policy

Phase 2.5B exists to prove the real vertical slice. Therefore the default development/integration path must use the real .NET backend.

- MSW/mock handlers may remain for isolated frontend tests or Storybook-like scenarios if already used.
- Do not leave production/development runtime silently defaulting to mock Opportunity APIs.
- A developer must be able to clearly tell whether the frontend is connected to the real backend.
- Do not create mock response shapes that diverge from generated/actual backend contracts.

21. Type Safety and API Contracts

Prefer generated or centrally defined API types if the repository already has an OpenAPI client-generation strategy. Do not duplicate DTO definitions in multiple frontend files.

- One canonical frontend representation per backend contract.
- No any-casting around authorization or lifecycle values.
- Do not convert configurable pipeline stages into TypeScript enums.
- Canonical lifecycle values may be typed according to backend contract.
- ProblemDetails should have a single parsing/mapping layer.

22. Customer Need - Preserve, Do Not Expand

CustomerNeed / OpportunityNeed is strategically important but full Customer Need Intelligence belongs to Phase 3.

In Phase 2.5B:

- Do not delete or hide backend support because it is not yet feature-complete.
- If Phase 2 already exposes read-only need data required for Opportunity detail, it may be displayed minimally.
- Do not implement AI voice inference, product inference, confidence, confirmation, or potential-value editing unless already binding in Phase 2.
- Do not conflate Need Potential with Opportunity value or future Quote total.

23. Explicit Non-Goals

- Customer Need Intelligence / AI inference.
- Activity Timeline, Calls, Meetings, Notes, Email sync.
- Quote, Quote Version, Quote public link, Quote tracking.
- Sales Order, Invoice, Refund, Payment.
- Feedback / employee or tenant evaluation.
- KVKK / consent management UI.
- Advanced analytics, forecasting, dashboards.
- RAG, LangGraph, AI agents, next-best-action.
- Admin IAM console redesign.
- Global shell redesign.
- Backend domain redesign unless a concrete blocking defect is found.

24. End-to-End Test Matrix

Use Playwright for browser E2E and real backend integration where the repository supports it. Reuse Phase 2.5A auth helpers/fixtures.

Test	Expected result
Login -> CRM	Authenticated user reaches Opportunity list.
Create	Valid create succeeds and detail opens.
Validation	Invalid create/update preserves form and shows errors.
Stage	Only allowed target transition is selectable and succeeds.
Forbidden stage	Direct browser/API attempt is denied; UI cannot bypass.
Reassign	Authorized reassignment succeeds; unauthorized one is unavailable/denied.
Win	Authorized Win succeeds and terminal state renders.
Lose	Authorized Lost succeeds with required reason semantics.
Field READ	Restricted field is omitted/masked correctly.
Field WRITE	Restricted field cannot be edited and direct request is denied.
Concurrency	Second stale browser/session receives conflict; no overwrite.
Idempotency	Double click/network replay does not duplicate logical mutation.
Tenant switch	Old tenant Opportunity data disappears; queries are isolated.
401 recovery	Expired access token restores through Phase 2.5A refresh path.
403/404	No cross-tenant existence leakage through frontend handling.

25. Component and Integration Tests

- Form validation and API error mapping.
- Permission/action rendering from backend capability projection.
- Field read/write behavior.
- Stage selector driven by allowed transitions.
- Mutation idempotency-key lifecycle.
- Concurrency conflict UI.
- Tenant-switch cache invalidation.
- ProblemDetails mapping.
- Loading/empty/forbidden/not-found states.

26. Security Review Checklist

- No auth access token persistence reintroduced by CRM code.
- No tenant id editable or trusted from arbitrary form data when session context should supply it.
- No frontend-only authorization assumption.
- No sensitive field leaked in cached state, console logs, error payload rendering, or analytics telemetry.
- No cross-tenant query cache reuse after tenant switch.
- No arbitrary principal enumeration during reassignment.
- No domain command issued using stale version without server conflict handling.

- No idempotency key logged as sensitive request data if policy forbids it.
- No raw internal ProblemDetails stack/exception exposed to users.

27. Recommended Implementation Sequence

1. Inspect repository and write PHASE_2_5B_CRM_FRONTEND_IMPLEMENTATION_PLAN.md.
2. Verify real API connectivity and disable accidental default mock path.
3. Create canonical Opportunity API client/query layer and ProblemDetails mapper.
4. Implement Opportunity List.
5. Implement Opportunity Detail read-only state and available-actions/field-access consumption.
6. Implement Create Opportunity.
7. Implement Edit Opportunity using field WRITE and concurrency contracts.
8. Implement Change Pipeline Stage.
9. Implement Reassign.
10. Implement Win and Lose.
11. Implement concurrency/idempotency/error UX hardening.
12. Complete component/integration tests.
13. Complete Playwright end-to-end test matrix against real backend.
14. Perform security review and final cleanup.
15. Produce PHASE_2_5B_CRM_FRONTEND_FINAL_REPORT.md and stop.

28. Open Decision Protocol

Do not silently invent product/domain decisions. Stop only when a missing decision blocks correct implementation.

For every blocker, write:

```
OPEN DECISION
- Issue
- Existing repository evidence
- Options
- Architectural impact
- UX impact
- Migration/API impact
- Recommendation
- Why implementation cannot safely continue without approval
```

Examples that may be blockers only if Phase 2 did not already decide them:

- No authoritative way to discover allowed actions/transitions.
- No eligible-assignee query but Reassign is required.
- Ambiguous field-level read/write contract.
- Ambiguous concurrency contract (rowVersion vs ETag/If-Match).
- No defined Lost reason contract.
- No defined initial pipeline/entry stage semantics.
- Inconsistent 403 vs 404 behavior for record-level denial.

29. Final Report

At completion create:

```
docs/architecture-analysis/PHASE_2_5B_CRM_FRONTEND_FINAL_REPORT.md
```

The final report must contain:

- Implemented routes/screens/components.
- Actual API endpoints consumed.
- Authorization/FLS/obligation integration.

- Concurrency and idempotency behavior.
- Tenant-switch/cache-isolation behavior.
- Error-state implementation.
- Tests added and passing status.
- Security review results.
- Any deferred items with reasons.
- Known limitations.
- Phase 3 readiness statement.

30. Acceptance Criteria - Binding

Area	Pass condition
Authentication	Phase 2.5A session/tenant model is reused; no parallel auth.
List	Real backend data is shown with tenant-safe behavior.
Create	Opportunity can be created through real Phase 2 API.
Detail	Record loads with lifecycle, stage, field access, and allowed actions.
Edit	Only authorized fields are editable and saved safely.
Stage	Only backend-allowed transitions can be executed.
Reassign	Respects eligible principals and authorization.
Won/Lost	Dedicated commands work; terminal state is represented correctly.
FLS	READ and WRITE restrictions are respected in the browser.
Obligations	RequireApproval/obligations are not collapsed into boolean allow.
Concurrency	Stale writes never silently overwrite newer data.
Idempotency	Duplicate logical submits do not duplicate business effects.
Errors	401/403/404/409/validation/domain errors have correct UX.
Tenant switch	No stale cross-tenant Opportunity data remains.
Mocks	Real API is the default integration path for this phase.
Tests	Required component/integration/E2E tests pass.
Scope	No Phase 3+ capabilities are accidentally implemented.

31. Final Console Output

After implementation and tests, print only a concise final summary in this shape:

PHASE 2.5B COMPLETE

- Routes/screens implemented:
- Real API integration:
- Authorization/FLS integration:
- Available actions/transitions:
- Concurrency behavior:
- Idempotency behavior:
- Tenant isolation/cache behavior:

- Tests:
- Security review:
- Deferred items:
- Phase 3 readiness: READY / NOT READY

32. Stop Condition

After the final report, STOP. Do not begin Phase 3 Customer Need Intelligence. Do not start Sales/Quote, Activities, Feedback, Consent, or AI work.